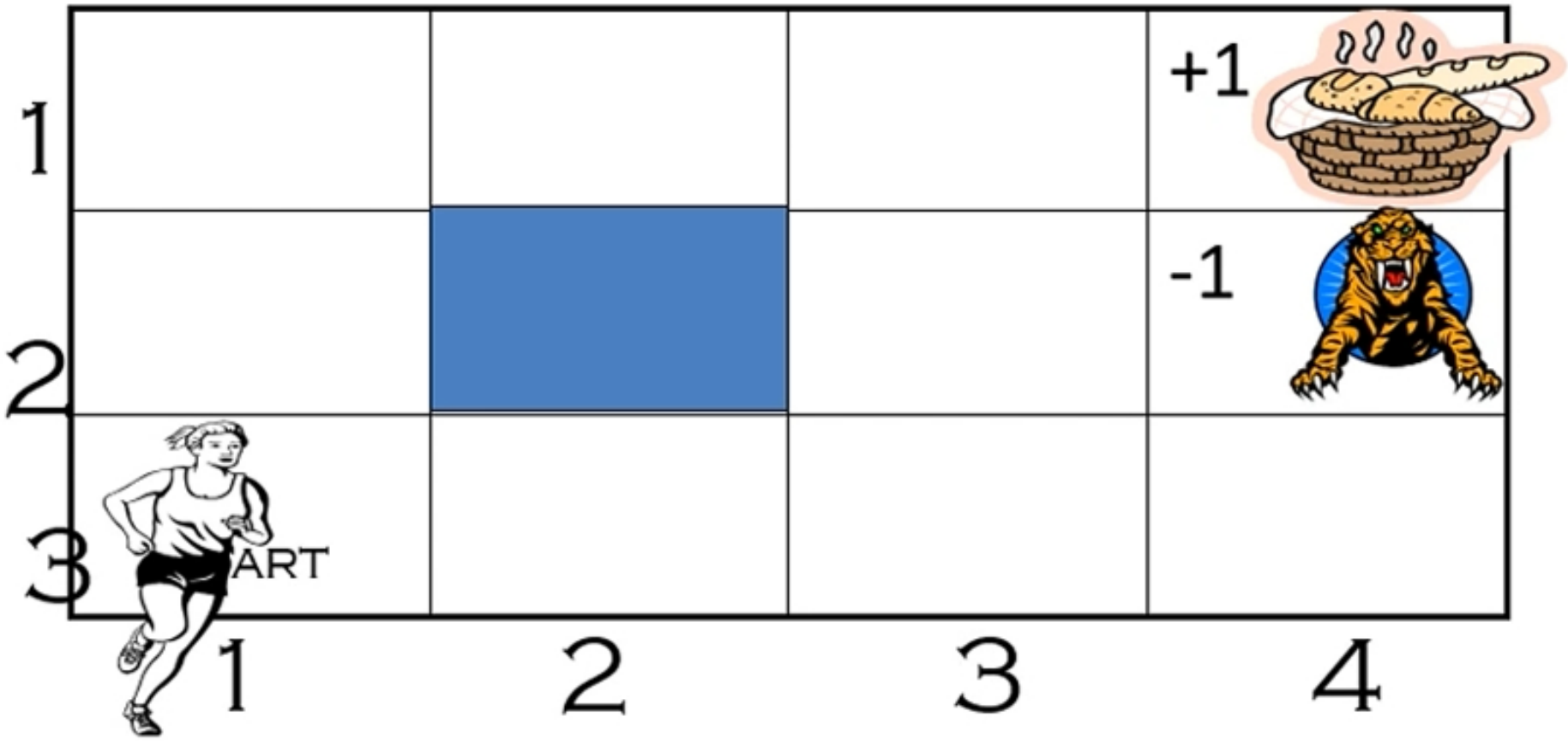
# Computing Optimal Policies

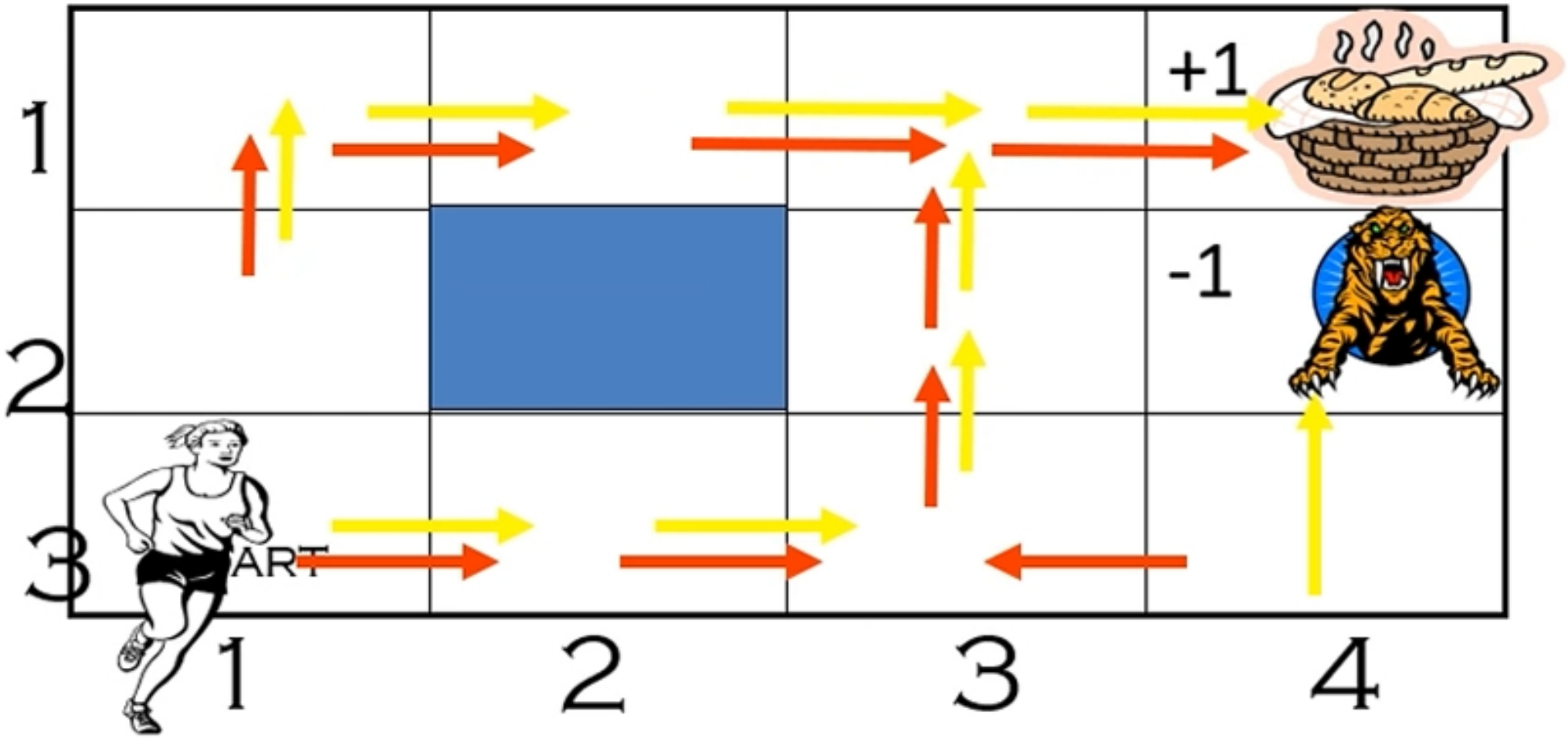
We will look at following three techniques:

1. **Value Iteration**
2. **Policy Iteration**
3. **Linear programming**

# Non-Optimal Vs Optimal Policy

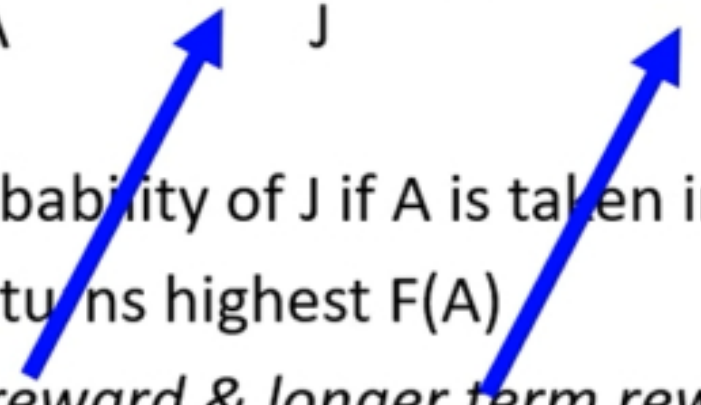


# Non-Optimal Vs Optimal Policy



# Value Iteration: Key Idea

- **Iterate:** Update utility of state "I" using **old utility** of neighbor states "J"; given actions "A"
  - $U_{t+1}(I) = \max_{A} [R(I,A) + \sum_{J} P(J|I,A) * U_t(J)]$
  - P(J|I,A): Probability of J if A is taken in state I
  - max F(A) returns highest F(A)
  - *Immediate reward & longer term reward taken into account*



- Dynamic programming

*Note: Slight change over textbook because use reward R(I,A) instead of R(I), but equivalent!*

# Value Iteration: Algorithm

- *Basic algorithm is very simple!*
- *Initialize:* $U_0(I) = 0$
- Iterate:

$$U_{t+1}(I) = \max_{A} \left[ R(I,A) + \sum_{J} P(J|I,A) * U_t(J) \right]$$

–Until close-enough $(U_{t+1}, U_t)$

*Dr. Richard Bellman*

- *Iteration step called "Bellman update"*
- *Inventor of dynamic programming (1957)*

# Value Iteration

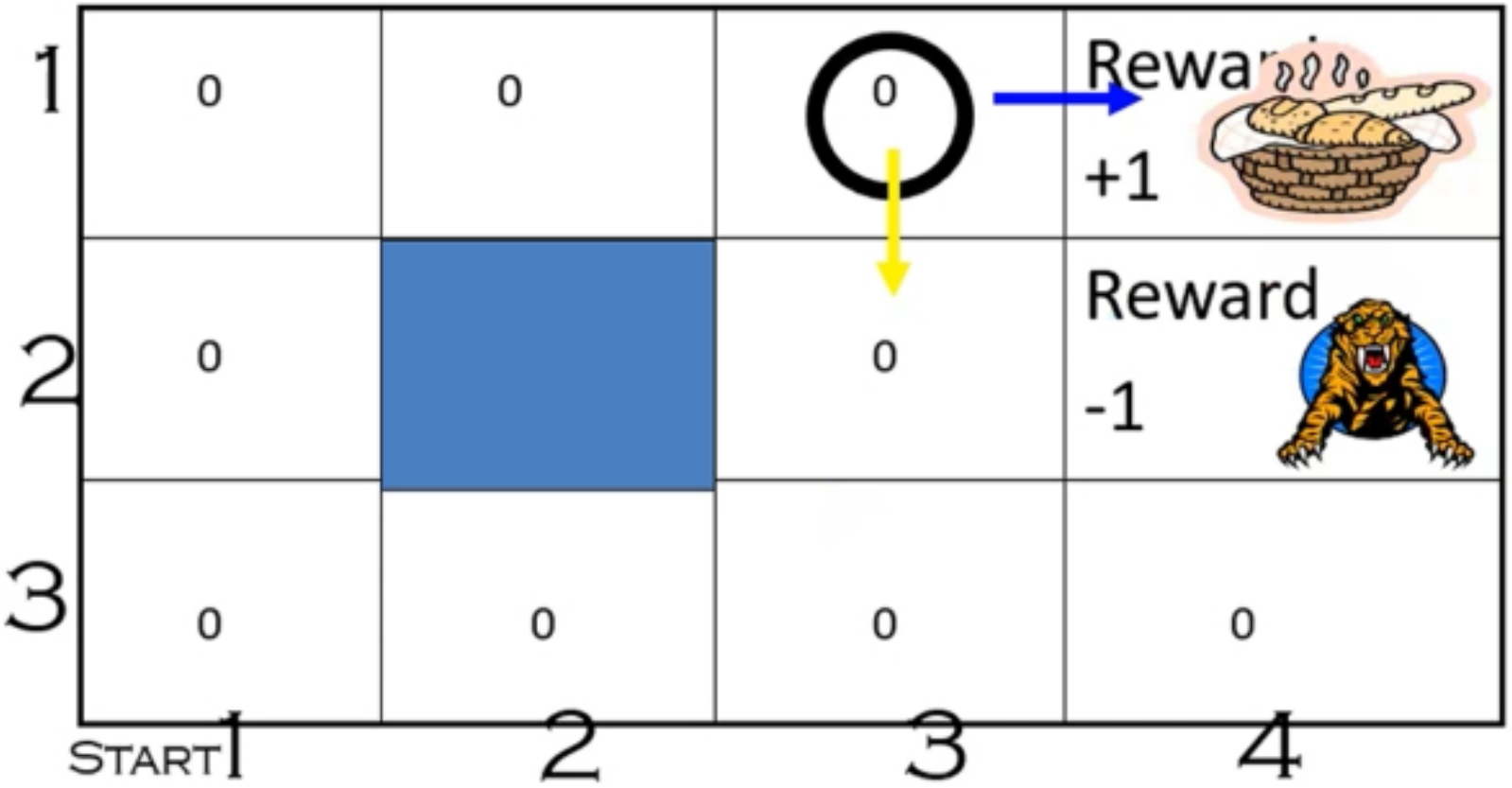
- How to determine close enough:
  - Let $\delta$ = max change in utility of any state in an iteration
  - $\delta$ <= BOUND

- At the end of iteration, calculate optimal policy:

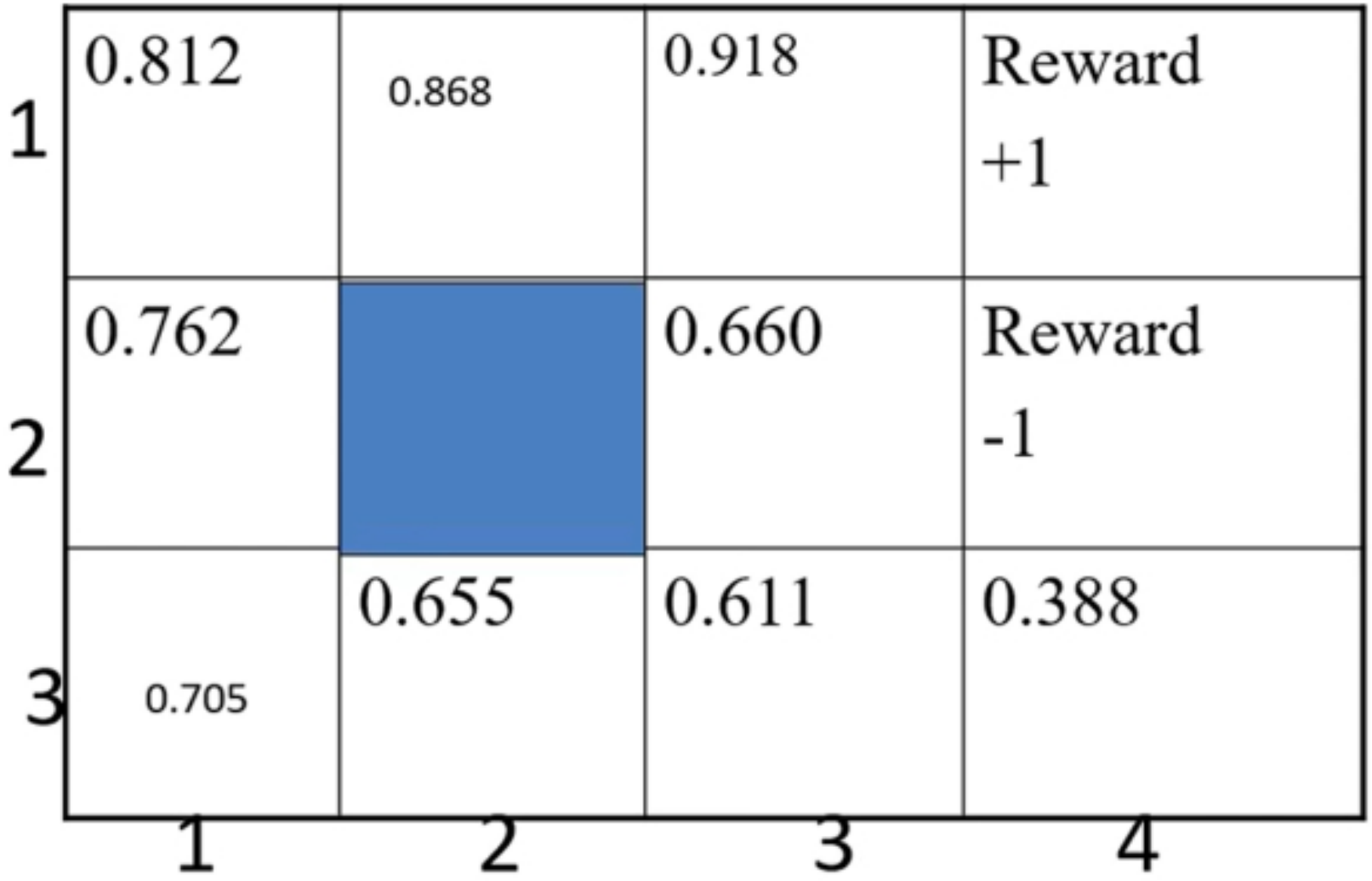
$$Policy(I) = \underset{A}{argmax} \left[ R(I,A) + \sum_{J} P(J \mid I,A) * U_{t+1}(J) \right]$$

# Iteration #1: Cell (3,1)

- East: -1/25 + [ 0.8 * 1 + 0.1 * 0 + 0.1 * 0 ] = 0.76
- North/South: -1/25 + [0.8 * 0 + 0.1 * 1 + 0.1 * 0 ] = 0.06

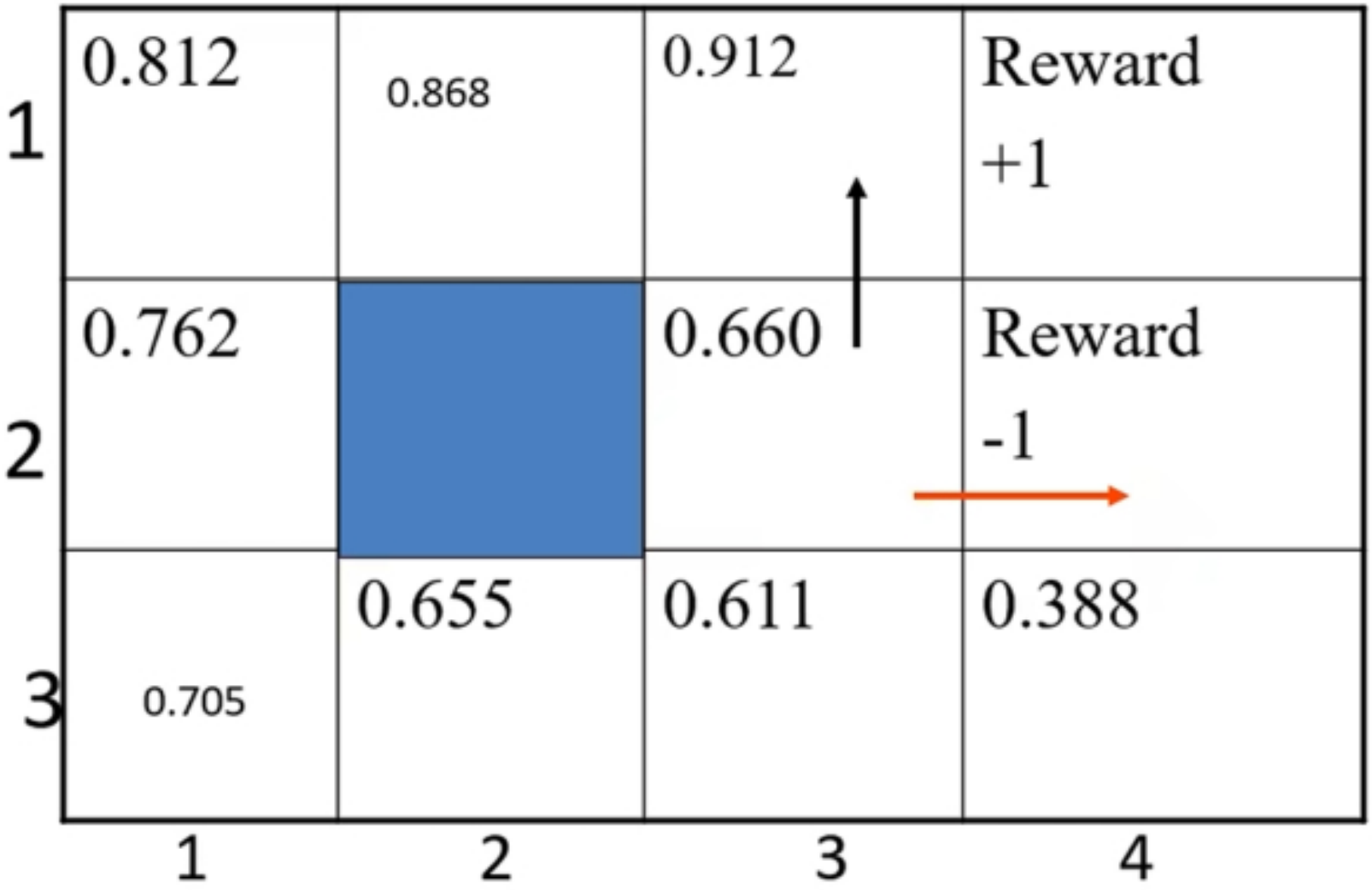


# Final Result of value iteration from Textbook




Why is the value of state (3,1) not 1? Why is it 0.918?

# Example



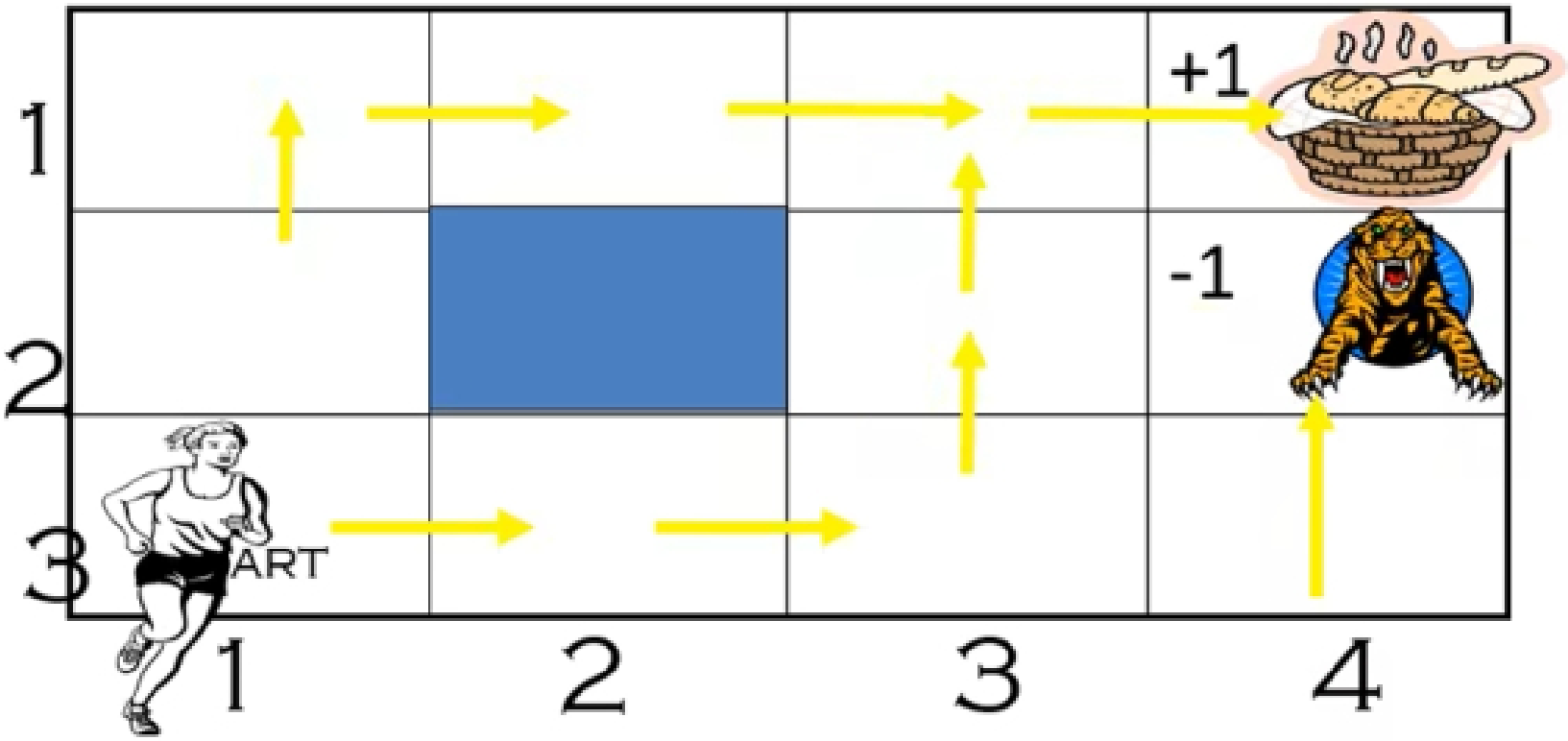

EU for Red action: -1/25 + [0.8 * -1 + 0.1* .611 + 0.1*0.912]
EU for White action: -1/25 + [0.8 * 0.912 + 0.1*0.660 + 0.1 * -1]
Which is a better action in cell (3,2)?

# Policy Notation

- *Policy typically denoted by* $\pi$
- *Optimal policy denoted by* $\pi^*$

- $\pi^*(s)$ = Optimal action in state s given $\pi^*$
- $U^{\pi^*}(s)$ = Optimal value of a state if the agent executes the optimal policy

# So, which policy is better ?




- Choose Red policy or Yellow policy?
- Choose Red policy or Blue policy?

How can we compare two policies?

# Markov Chain

- Given fixed policy, you get a **Markov chain** from the MDP
  - **Markov chain:** Next state is dependent only on previous state
  - **Next state:** Not dependent on action (there is only one action)
  - **Next state:** History dependency only via the previous state

  - $P(S_{t+1} \mid S_t, S_{t-1}, S_{t-2} \ldots\ldots) = P(S_{t+1} \mid S_t)$

- How to evaluate the Markov chain?

# Evaluate Markov Chain

- *Use **value iteration algorithm**, but with fixed action*

---

- *Initialize: $U_0(I) = 0$*
- Iterate:

$$U_{t+1}(I) = R(I,A) + \sum_J P(J|I,A) * U_t(J)$$

- Until close-enough ($U_{t+1}$, $U_t$)

---

To compare two policies:

- Check the value of start state (1,3) given the two evaluations obtained above

# Discounting

- If agents continue to live forever, the rewards they accumulate would be **infinite**
- How to compare multiple policies in such cases?

- **Discounting:** Rewards at future time-steps are less valuable than in the current time step, with a factor $\gamma$
- $0 <= \gamma < 1$
- Reward at a future time step i discounted by $\gamma^{i}$

- $\gamma$ controls effect of future rewards on current decisions
- We may typically take $\gamma$ to be 0.95

# Value Iteration: Modify

- *Initialize:* $U_0(I) = 0$
- Iterate:

$$U_{t+1}(I) = \max_{A} \left[ R(I,A) + \gamma \sum_{J} P(J|I,A) * U_t(J) \right]$$

  – Until close-enough $(U_{t+1}, U_t)$

- At the end of iteration, calculate optimal policy:

$$Policy(I) = \operatorname*{argmax}_{A} \left[ R(I,A) + \gamma \sum_{J} P(J|I,A) * U_{t+1}(J) \right]$$

# Policy Iteration

- Start with some (random) policy
- Repeat until policy stabilizes (no change)
  - Determine value of policy $\pi$ (see previous lecture)
    - *Provides us a value of each state I given policy $\pi$*
  - At each state I do:
    - If /* Check if policy can be improved at each state */

$$\max_{A}\left[ R(I, A) + \sum_{J} P(J \mid I,A)* U_t(J) \right] >$$

$$R(I,\text{Policy}(I)) + \sum_{J} P(J \mid I, \text{Policy}(I))* U_t(J)$$

# Linear Programming

- Mathematical programming is used to find the **best or optimal solution** to a problem that requires a decision or set of decisions about how best to use a set of limited resources to achieve a state goal of objectives.
- **Steps involved in mathematical programming**
  - Conversion of stated problem into a mathematical model that abstracts all the essential elements of the problem.
  - Exploration of different solutions of the problem.
  - Finding out the most suitable or optimum solution.

# The Linear Programming Model (1)

Let: $X_1, X_2, X_3, \ldots\ldots\ldots, X_n$ = decision variables

$Z$ = Objective function or linear function

Requirement: Maximization of the linear function Z.

$$Z = c_1X_1 + c_2X_2 + c_3X_3 + \ldots\ldots\ldots + c_nX_n \qquad \ldots\ldots\text{Eq (1)}$$

subject to the following constraints:

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &\leq b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &\leq b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &\leq b_n \\
\text{all } x_j &\geq 0
\end{aligned}$$